Session 10 Lab

Complete Kotlin Code

```
// 🧩 Session 10 Lab - Product Analyzer with Higher-Order Functions
// ---------------------------------------------------------------

// TODO 1: Define Product class
data class Product(
    val name: String,
    val category: String,
    val price: Double,
    val stock: Int
)

// TODO 2: Higher-order function to apply discount
fun applyDiscount(
    products: List<Product>,
    discountFunction: (Double) -> Double
): List<Product> {

    return products.map { product ->
        product.copy(price = discountFunction(product.price))
    }
}

// TODO 3: Higher-order function to filter products
fun filterProducts(
    products: List<Product>,
    condition: (Product) -> Boolean
): List<Product> {

    return products.filter { product ->
        condition(product)
    }
}

// Function to print product list neatly
fun printProducts(products: List<Product>) {
    for (product in products) {
        println(
            "Name: ${product.name} | " +
            "Category: ${product.category} | " +
            "Price: ${product.price} | " +
```

```
            "Stock: ${product.stock}"
        )
    }
}

fun main() {

    // TODO 4.1: Create sample product list
    val productList = listOf(
        Product("Laptop", "Electronics", 100000.0, 5),
        Product("Phone", "Electronics", 50000.0, 15),
        Product("Chair", "Furniture", 2000.0, 25),
        Product("Table", "Furniture", 3000.0, 5),
        Product("Pen", "Stationery", 50.0, 100)
    )

    // TODO 4.2: Apply 10% discount
    val discountedProducts = applyDiscount(productList) { price ->
        price * 0.9
    }

    println("--- Discounted Product List ---")
    printProducts(discountedProducts)

    // TODO 4.3a: Products with stock < 10
    val lowStockProducts = filterProducts(productList) { product ->
        product.stock < 10
    }

    println("\n--- Low Stock Products (<10) ---")
    for (product in lowStockProducts) {
        println("${product.name} (${product.stock})")
    }

    // TODO 4.3b: Products with price > 1000
    val premiumProducts = filterProducts(productList) { product ->
        product.price > 1000
    }

    println("\n--- Premium Products (>1000) ---")
    for (product in premiumProducts) {
        println("${product.name} (${product.price})")
```

```
    }
}
```

How This Lab Meets the Requirements

✓ Uses Class

data class Product stores structured product data.

✓ Uses Higher-Order Functions

- applyDiscount() takes a lambda (Double) -> Double
- filterProducts() takes a lambda (Product) -> Boolean

✓ Uses Collection Transformations

- map() → Transform price
- filter() → Select specific products

✓ Immutability

Original product list is not modified.
copy() creates a new Product object with updated price.

Sample Output (Example)

--- Discounted Product List ---
Name: Laptop | Category: Electronics | Price: 90000.0 | Stock: 5
Name: Phone | Category: Electronics | Price: 45000.0 | Stock: 15
Name: Chair | Category: Furniture | Price: 1800.0 | Stock: 25
Name: Table | Category: Furniture | Price: 2700.0 | Stock: 5
Name: Pen | Category: Stationery | Price: 45.0 | Stock: 100

--- Low Stock Products (<10) ---
Laptop (5)
Table (5)

--- Premium Products (>1000) ---
Laptop (100000.0)
Phone (50000.0)
Chair (2000.0)
Table (3000.0)

Short Reflection

This lab helped me understand how higher-order functions improve flexibility and reduce code duplication. Using map() and filter() makes collection processing more readable and efficient. Lambdas allow dynamic behavior without rewriting logic. Overall, this approach improves maintainability and functional programming understanding in Kotlin.